JNEOPALLIUM · DEMO 06

ARCHITECTURE ARTICLE

A Digital Immune System for Cybersecurity

Temporal Threat Correlation — Complete Architecture, Explained

How a brain- and immune-system-inspired engine spots multi-step cyber attacks that traditional, single-alert tools miss — written so a non-specialist can follow every step.

Document	Architecture & Technical Overview
Product	Jneopallium Cybersecurity Module (Demo 06)
Model	cybersecurity-temporal-threat-correlator 1.0.0
Safety mode	ADVISORY (recommend-only)
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	23 June 2026
Audience	Executives, security leads, engineers, newcomers
License	BSD 3-Clause

What's inside

1 Executive summary

Most security tools look at one event at a time. They ask, "Is this single login, this single file, this single network connection dangerous?" Real attackers do not work that way. They move in **quiet, patient steps**: a suspicious login, then a script, then a lookup, then a jump to another machine, then a slow trickle of data leaving the building. Each step on its own looks almost normal. The damage is in the **sequence**.

This product — Demo 06 of the Jneopallium platform — is built to watch the sequence. It is a **temporal threat-correlation engine**: it connects events across time and across many different data sources, recognises the shape of an attack as it unfolds, and raises a single, well-evidenced advisory instead of a thousand disconnected alarms.

Its design is borrowed from the human **immune system** and the human **brain**. Just as your body has fast first-responders, slower learning defences, a memory of past infections, and a hard rule never to attack its own healthy cells, this engine has fast detectors, adaptive learning, an attack memory, and inviolable safety gates that protect critical systems.

> **The one sentence to remember**
>
> It does not just ask "is this event bad?" — it asks "do these events, in this order, over this stretch of time, add up to an attack?" — and it answers with evidence, never with a blind block.

Crucially, the system runs in **ADVISORY mode**: it recommends, it never silently isolates a host or blocks traffic on its own. Every recommendation carries a full evidence trail. That makes it safe to deploy alongside the tools you already trust.

2 The problem: why modern attacks slip through

Three weaknesses of one-event-at-a-time tools

Classic intrusion-detection systems score each event in isolation. This creates three well-known failures that attackers exploit on purpose:

- **The low-and-slow blind spot.** An attacker who steals data a little at a time — a few megabytes an hour over days — never trips a single big threshold. Each transfer looks like ordinary traffic.
- **Alert fatigue.** When every event is scored alone, a busy network produces thousands of low-confidence alerts a day. Analysts cannot read them all, so the real attack hides in the noise.
- **Lost context.** A PowerShell script during an approved maintenance window is routine. The exact same script at 3 a.m. on a finance server, right after an unusual login, is an emergency. A single-event tool cannot tell the two apart.

What "dwell time" costs

The industry term for how long an attacker stays undetected is **dwell time**. The longer the dwell time, the more credentials are stolen, the more machines are compromised, and the more data leaves. Every hour of earlier detection directly reduces the blast radius. The entire purpose of temporal correlation is to **collapse dwell time** by recognising the attack while it is still in progress, not after the data is gone.

3 The big idea: borrowing from biology

Your immune system is the most successful intrusion-detection system on Earth. It has been field-tested for hundreds of millions of years. Jneopallium's cybersecurity module deliberately copies its architecture, because the problems are the same: detect threats fast, learn new ones, remember old ones, respond in proportion, and — above all — never destroy healthy tissue.

Biological defence	What it does	In this product
Macrophages / neutrophils	Fast, hard-wired first responders that recognise known danger signs	**Innate signature detectors** — match known attack patterns instantly
T-cells	Adaptive defenders that learn what is abnormal for *your* body	**Anomaly detectors** — learn each user's and host's normal behaviour
Memory B-cells	Remember past infections so the next response is faster	**Attack memory** — recalls campaigns and techniques seen before
Thymic negative selection	A hard rule: never attack the body's own healthy cells	**Hard safety gate** — fixed allow-list, never learned, protects critical assets
Inflammation staging	Response scales up gradually, never all-or-nothing	**Graduated response** — log → alert → quarantine candidate → escalate
Resolution of inflammation	Defences stand down so tissue can heal	**Auto-lift** — every quarantine has a deadline and expires automatically

Two of these analogues deserve special attention because they are what make the system trustworthy:

- **Self-tolerance (never attack yourself).** The biological immune system is dangerous when it turns on healthy cells — that is what auto-immune disease is. The product's hard safety gate is the digital equivalent of negative selection: a fixed, construction-time list of protected systems that **no runtime signal can override**. An attacker who somehow gets inside the data flow can, at most, un-block something previously trusted — they can never turn the system into a weapon against your own critical servers.
- **Resolution (defences always stand down).** Inflammation that never resolves kills the patient. So in this product, **quarantine is never permanent by construction**: every containment recommendation carries a positive time limit and an automatic lift fires when it expires, unless the threat is independently re-confirmed.

4 The platform underneath: the Jneopallium framework

The cybersecurity module is one application of a general-purpose engine called **Jneopallium** — a Java framework for building biologically-grounded neuron networks, published in the *International Journal of Science and Research* (2024). You do not need to understand neuroscience to use the product, but three of the framework's ideas explain *why* it is good at this job.

1. Typed signals — a programmable nervous system

In an ordinary neural network, everything is a number. In Jneopallium, you define what a **signal** is. A network packet, a login event, a threat-intelligence indicator, and a maintenance notice are *different kinds of signal* with different meaning and different urgency. The engine routes each kind to the right specialist. This is exactly why the system can keep evidence — it never flattens a login and a packet into the same anonymous number.

2. Two clocks — fast reflexes and slow judgement

Biology runs on many clocks at once: nerve impulses travel in milliseconds, while hormones diffuse over seconds to minutes. Jneopallium copies this with a **fast loop** (every tick) and a **slow loop** (every N ticks). Network packets and system calls are processed on the fast loop for instant reaction; slower context like threat intelligence, asset criticality, and maintenance windows is processed on the slow loop. The result: detection stays fast, while context that should *modify* a verdict is applied at its own natural pace.

3. Stateless, swappable specialists

Each processing step is a small, stateless **processor** wired to a neuron through an interface, not a hard-coded class. In practice this means any detector — the signature engine, the anomaly model, the correlation logic — can be upgraded or replaced for a particular deployment without touching the rest of the system. A lab can run a simple matcher; a bank can plug in a high-performance commercial engine behind the same interface.

5 Architecture overview: how the pieces fit together

At the highest level, telemetry flows in one direction through a pipeline of increasingly intelligent stages, and an auditable advisory flows out the other end:

```
multi-source telemetry
   -> canonical event adapters      (translate everything into one language)
   -> fast host & network receptors  (cheap, instant evidence scoring)
   -> temporal threat correlation    (connect events over time)
   -> response planner               (choose a proportionate recommendation)
   -> fixed hard safety gate         (protect critical assets, enforce mode)
   -> advisory output + audit trail  (a verdict you can explain)
```

The same shape appears inside the runnable demo as a small four-layer neuron network. Each layer is a team of neurons with a clear job:

Layer	Size	Job, in plain language
Layer 0 — Input	7	Receive seven kinds of raw security telemetry
Layer 1 — Normalisation	4	Clean it up and give each event a fast evidence score
Layer 2 — Correlation	3	Connect events across time into a threat hypothesis
Layer 3 — Planning	2	Turn the hypothesis into an advisory investigation action

The full production module is richer — it adds memory, homeostasis (self-regulation), and a dedicated response layer — described next.

6 The seven layers, explained in plain language

The production cybersecurity module is organised like the body's defences, from the skin inward. Here is each layer and the everyday job it does.

Layer 0 — Ingestion (the senses)

Rate-limited intake of packets, system calls, and logs. Like a doorway with a turnstile, it accepts telemetry at a controlled rate so a flood of traffic — accidental or a deliberate denial-of-service attempt — cannot overwhelm everything behind it.

Layer 1 — Innate detection (the first responders)

Fast, hard-wired matchers: known-bad signatures, forbidden sequences of system calls, and per-connection traffic accounting. A **soft allow-list** here filters known-good patterns. This layer is cheap and instant — it catches the obvious.

Layer 2 — Adaptive detection (the learners)

This is where the system learns *your* normal. It keeps a moving baseline of each user's and host's behaviour and flags meaningful deviation. It includes a **beaconing detector** (spotting the metronome-like regularity of malware phoning home) and a **lateral-movement detector** (spotting one account suddenly authenticating to many machines). Critically, this layer's learning **freezes during an active attack**, so the attacker's behaviour is never quietly absorbed into the definition of "normal."

Layer 3 — Memory (the immune memory)

Remembers attack campaigns and the techniques that made them up, and binds scattered pieces of evidence onto a single incident timeline. This is how a faint new signal can be recognised quickly as part of a pattern the system has seen before.

Layer 4 — Hypothesis & planning (the diagnosis)

Combines signature evidence and anomaly evidence into a single probability — a **posterior** — that an attack is underway, then maps that probability to a proportionate response band. This is the layer that turns scattered clues into a verdict with a confidence level.

Layer 5 — Response (the proportionate reaction)

Owns the **hard safety gate** (the un-overridable allow-list and critical-asset protection), the **quarantine logic** (positive-duration only, automatic lift), and optional snapshot rollback. In ADVISORY mode this layer produces *recommendations and evidence*, not enforced actions.

Layer 7 — Homeostasis (self-regulation)

Keeps the system healthy. An **alert-fatigue monitor** raises detection thresholds when false positives drift up, and an **exhaustion limiter** acts as a budget on rule evaluation so a flood cannot starve the important stages. Biology calls this homeostasis; engineering calls it graceful degradation under load.

> **Why the layers are numbered 0–7 with a gap**
>
> The numbering mirrors the broader Jneopallium cognitive architecture, where some layers (such as affect and curiosity) are deliberately switched off for security work — a defensive system should not get "bored" or "emotional." The gaps are intentional, not missing pieces.

7 Signals: the common language of the system

Everything the system sees is converted into a **typed signal** — a small, well-described object carrying both data and meaning. This is what lets the engine keep an evidence trail from raw telemetry all the way to the final advisory. The cybersecurity demo defines ten signal types:

Signal	Carries	Think of it as
AuthenticationEventSignal	Logins, their result and context	Who tried to get in, and did it work
ProcessEventSignal	Program execution, parent/child chains	What ran, and what launched it
DnsLookupSignal	Domain-name lookups	Who the machine tried to phone
NetworkFlowSignal	Connection byte/packet summaries	How much data went where
ThreatIntelContextSignal	Known-bad indicators with confidence	An external tip-off
AssetContextSignal	How critical and owned an asset is	How much this machine matters
MaintenanceWindowSignal	Approved-change context	"This was planned" context
RiskScoreSignal	A computed risk number	The running risk tally
ThreatHypothesisSignal	A correlated attack hypothesis	The diagnosis-in-progress
SecurityAdvisorySignal	The final recommendation + evidence	The verdict you act on

Each signal type also declares **how often** it should be processed. Fast, urgent signals (packets, system calls) run every tick; slower context signals (threat intel, maintenance, self-tolerance updates) run on the slow loop. This is the "two clocks" idea made concrete: the system reacts instantly but reflects slowly, exactly where each is appropriate.

8 Temporal correlation: connecting the dots over time

This is the heart of the product. A real intrusion is not one event — it is an ordered chain. The classic enterprise attack looks like this:

```
unusual authentication
      ->  process execution
            ->  DNS lookup
                  ->  lateral movement (jump to another host)
                        ->  command-and-control beaconing
                              ->  data exfiltration
```

The engine watches several overlapping time windows at once, each tuned to a different rhythm of attacker behaviour:

Window	Span	What it catches
Fast window	1–10 seconds	Immediate, sharp signals
Behaviour window	1–5 minutes	A burst of related actions
Incident window	30–120 minutes	A full attack chain unfolding
Baseline window	Hours to days	What "normal" looks like over time

Within these windows the model rewards **ordered transitions** — a login *followed by* an execution *followed by* lateral movement scores far higher than the same events in a random order or spread randomly across unrelated machines. It also recognises the **low-and-slow** pattern: weak, repeated

outbound transfers that mean nothing individually but, correlated over an incident window and combined with threat-intelligence context, reveal a quiet data leak.

> **Why event *time* matters more than arrival time**
>
> Telemetry arrives late, out of order, and replayed. The engine correlates by the time an event *actually happened* (its event-tick), not the time it was received. That is what lets it reconstruct the true sequence of an attack even when the data is messy — a property single-event tools lack.

9 The trained model: what it actually learned

Bundled with the product is a checked-in reference model, `cybersecurity-temporal-threat-correlator`. It is deliberately a **transparent, reproducible baseline**: a class-balanced logistic model over 34 temporal-window features. Transparency is a feature — every weight is inspectable, so a security team can see *why* a verdict was reached before moving on to larger sequence models (GRU/TCN/transformer) later.

The model condenses each time-window into 34 numbers (features) such as: how ordered the events are, how diverse the data sources are, the strongest threat-intelligence hit, the average asset criticality, and whether a maintenance window was active. The features it leaned on most tell a coherent story:

Top evidence the model treats as suspicious

Feature	Weight	Plain meaning
technique_command_and_control	+0.43	Signs of a machine phoning home to an attacker
network_receptor_score	+0.42	Strong network-side evidence
max_threat_intel	+0.42	A high-confidence external threat tip
slow_context_score	+0.39	Threat intel plus asset criticality together
mean_evidence	+0.39	Consistently strong evidence across the window

Top evidence the model treats as reassuring

Feature	Weight	Plain meaning
maintenance_ratio	-0.45	Activity happened during approved maintenance
benign_context_ratio	-0.39	Mostly benign, expected context
technique_unusual_login	-0.27	An isolated odd login, with nothing following it
source_diversity	-0.18	Evidence from only one source, not a coordinated chain

From model to deployable network

The trainer does not stop at a set of weights — it emits a **complete, deployable JNeopallium network**: five layers of eight real neurons, with the learned weights, the decision threshold, the temporal sequence gates, and the safety policy all written into ready-to-load layer configuration files. What was trained is exactly what runs in production.

Generated layer	Real neurons	Job
Input	—	Multi-source event boundary
Fast evidence	NetworkFlowNeuron, SignaturePatternNeuron, ProcessBehaviourNeuron, EntityBehaviourBaselineNeuron	Instant scoring
Correlation	TemporalThreatCorrelationNeuron	Trained temporal correlation
Planning + gate	ResponsePlanningNeuron, ResponseGateNeuron	Advisory bands + fixed gate
Result	ResponsePlanningNeuron	Advisory output

The correlation layer also carries a **baseline-adaptation policy** that freezes learning when the posterior reaches 0.3 or signature confidence reaches 0.8, adapting only from trusted benign periods — the anti-poisoning rule, encoded directly into the deployable network.

> **Read this honestly**
>
> On the bundled, deterministic reference data the model scores perfectly (precision and recall of 1.0). That demonstrates the pipeline works end to end — it is NOT a claim of real-world accuracy. Real performance must be earned on external datasets. The accompanying Test Report is explicit about this distinction, and we keep it front-and-centre rather than buried.

10 Safety by design: why it cannot break your network

A detection system with the power to block traffic is also a system that can take your business offline if it is wrong. This product is engineered so that being wrong is *safe*. Four structural guarantees do the heavy lifting:

1. **Advisory by default.** The safety ceiling is ADVISORY. The system recommends investigation or containment candidates; it does not isolate hosts or block traffic on its own. Active enforcement requires a separate, deliberately added safety case, approval workflow, and rollback path.
2. **Hard gates are fixed, not learned.** The allow-list of protected systems and the critical-asset list are set at wiring time and cannot be changed by any runtime signal. The model can learn what looks suspicious; it can never learn to stand down protection of a critical server.
3. **Baseline freeze during attacks.** When the system's confidence reaches a watch/alert state, adaptive learning freezes, so an attacker cannot patiently teach the system that their behaviour is normal (a "baseline-poisoning" attack).
4. **Containment always expires.** Every quarantine recommendation has a positive duration and an automatic lift. Nothing the system recommends is permanent unless a human independently re-confirms it.

On top of these, every advisory preserves a full **evidence lineage**: the entity involved, the exact time range, which sources and techniques contributed, the posterior probability, the response band, whether the baseline was frozen, and the precise model version and checksum used. If a regulator, an auditor, or an analyst asks "why did the system say this?", there is always a complete, replayable answer.

11 Data foundation: the datasets behind the model

A temporal correlator is only as good as the variety of attacks it has seen. The training design deliberately spans seven complementary public and lab sources, each mapped onto the system's typed signals so the model learns from many angles rather than one narrow table:

Dataset	Role
LANL Comprehensive Multi-Source Cyber-Security Events	Realistic enterprise temporal validation with red-team labels
ToN_IoT	Multi-source Windows/Linux/network/IoT with ground truth
DARPA OpTC	Advanced endpoint and APT provenance validation
CIC-IDS2017 / CSE-CIC-IDS2018	Network-flow detector pre-training
UNSW-NB15	Fast network classifier and throughput validation
MITRE CALDERA lab output	Exact labels for controlled, repeatable attack chains

A strict **split policy** prevents the most common way machine-learning security claims are inflated: the data is never split by random individual rows (which leaks near-duplicate events of the same campaign into both training and testing). Instead it is split by time period, campaign, host group, and attack type — so the test always measures performance on genuinely unseen attacks.

12 Deployment topology: where it runs

The same model runs in three deployment shapes, so it fits a laptop demo and a clustered enterprise alike:

Mode	Description	Typical use
Local	Single Java process	Demos, pilots, edge sites
Cluster (HTTP)	Distributed across worker nodes over HTTP	Enterprise-scale telemetry
Cluster (gRPC)	Distributed over gRPC; FPGA targets supported	High-throughput, low-latency

Telemetry reaches the system through **bridges** — adapters that translate a real-world protocol into typed signals. The cybersecurity module is designed around a Kafka-style event stream, so a real Kafka bridge can replace the demo's input while keeping the exact same typed event contract. Because correlation is by event time, delayed or out-of-order telemetry still lands in the right place in the sequence.

13 End-to-end walkthrough: three streams, three verdicts

The clearest way to understand the system is to watch it judge three simultaneous streams that a naive tool would get wrong. All three run through the same pipeline; the difference is entirely in the **correlation and context**.

Stream A — the real attack (correctly raised)

`user:backup-service@workstation-17`: an unusual login, then an encoded PowerShell command, then authentication fan-out to many hosts, then a rare DNS lookup, then periodic outbound traffic. Individually, forgivable. In this **order**, within the incident window, it is a textbook intrusion. Verdict: **TEMPORAL_THREAT_ADVISORY**, with the baseline frozen so the attack cannot poison "normal."

Stream B — planned maintenance (correctly calmed)

`svc:deployment-agent@web-tier`: service-account retries and signed deployment activity during an approved maintenance window. The maintenance context **lowers the score** — but it does not erase the evidence; the system still records auditable observations. Verdict: **CONTEXT_SUPPRESSED_OBSERVATION**. This is the case single-event tools get most wrong, in both directions.

Stream C — the quiet data leak (correctly surfaced)

`host:finance-file-01`: weak, repeated outbound transfers that are individually meaningless. Correlated over the incident window and combined with threat-intelligence context, they become a credible slow exfiltration. Verdict: **LOW_AND_SLOW_CORRELATION** — precisely the case the low-and-slow blind spot was designed to exploit.

> **The point of the three streams**
>
> The attack scores higher than the benign maintenance; the maintenance is calmed without losing its audit trail; the quiet leak is surfaced. And in every case the output is an advisory with evidence — never a silent block. That combination is what a one-event-at-a-time tool cannot deliver.

14 Glossary for non-specialists

Term	Plain-language meaning
Advisory mode	The system recommends; humans decide. It never blocks on its own.
Anomaly detection	Spotting behaviour that is unusual for a specific user or machine.
Baseline	The learned picture of what "normal" looks like for an entity.
Baseline poisoning	An attacker slowly teaching a system that their behaviour is normal.
Beaconing	Malware contacting its controller at regular intervals, like a heartbeat.
Dwell time	How long an attacker stays in the network before being detected.
Evidence lineage	The full, replayable trail from raw data to a verdict.

Exfiltration	Stealing data out of the organisation.
Lateral movement	Jumping from one compromised machine to others.
Posterior	The model's probability that an attack is underway, given the evidence.
Quarantine	Temporarily containing an entity — here, always with an expiry.
Signature detection	Matching against known, named bad patterns.
Temporal correlation	Connecting events across time into one coherent picture.
Telemetry	The stream of raw security data: logins, processes, DNS, network flows.

Jneopallium Cybersecurity Module · Demo 06 · Temporal Threat Correlation. Architecture article prepared for technical and non-technical readers alike. Safety mode: ADVISORY. License: BSD 3-Clause.